

When Depth Adds Nothing: A Negative Result on Probing Recurrent-Depth Transformers

Anonymous ACL submission

Abstract

Chain-of-thought monitoring is increasingly recognized as a fragile alignment affordance whose adequacy is threatened by latent-reasoning architectures, including recurrent-depth transformers (RDTs), that perform iterative computation in continuous hidden-state space rather than in discrete tokens. A natural replacement candidate is to probe the depth dimension of the loop directly. We test this candidate by finetuning a 2.3M-parameter recurrent-depth transformer with Group Relative Policy Optimization on a task instrumented with an input-channel reward leak, and training linear probes at every loop depth on hidden states from the resulting RL-finetuned model. The task probes achieve AUROC = 1.0 at every depth. However, two pre-registered controls, probes on the pre-RL base model and probes on the input embedding layer alone, also achieve AUROC = 1.0 at every depth, and a single-bit feature indicating leak presence in the input achieves AUROC = 0.99. We conclude that for an input-channel exploit in this architecture, the recurrent loop does not contribute monitoring information beyond what is available from inspecting the input, and that input-layer baselines should be a mandatory control for any depth-probing study on a recurrent or looped architecture. We identify three exploit classes for which a positive depth-localization result would be expected and frame them as direct hypotheses for follow-up.

1 Introduction

Chain-of-thought (CoT) monitoring has emerged as a load-bearing affordance for modern alignment practice: by inspecting the natural-language reasoning a model produces before its final answer, an external monitor can flag misbehavior invisible at the output level (Baker et al., 2025; Korbak et al., 2025). Recent work establishes that this affordance is *fragile*. Training pressure that penalizes legible reasoning teaches models to obfuscate

their CoT while continuing to exhibit the underlying misbehavior (Baker et al., 2025), and a broad multi-laboratory position statement identifies the adoption of latent-reasoning architectures as a principal threat to CoT monitorability (Korbak et al., 2025).

That threat is no longer hypothetical. Continuous chain-of-thought systems (Hao et al., 2024), looped transformers (Saunshi et al., 2025; Bae et al., 2025), and depth-recurrent transformers (RDTs) scaled to billions of parameters (Geiping et al., 2025) perform iterative reasoning in continuous hidden-state space rather than in discrete tokens. For these architectures, no natural-language CoT exists to monitor. Wang et al. (2025) note explicitly that this property renders CoT-based monitoring methods structurally inapplicable, and Korbak et al. (2025) identify exactly this architectural class as the canonical case in which the CoT-monitoring window may close.

A natural question follows: does the depth dimension of an RDT—the sequence of hidden states $h_1, \dots, h_T$ produced by the recurrent loop—provide a monitoring affordance that substitutes for the missing natural-language reasoning trace? If reward hacking, the most empirically tractable misalignment-relevant behavior we know how to induce (Denison et al., 2024; MacDiarmid et al., 2025), leaves a *depth-localized* signature in the loop, then simple linear probes on h_t could in principle serve the same role for RDTs that CoT monitors serve for autoregressive reasoning models. This is the hypothesis we set out to test.

The negative result. We test the hypothesis directly. We finetune a recurrent-depth transformer with Group Relative Policy Optimization (Shao et al., 2024) on a task instrumented with a planted reward exploit, train linear probes at every loop depth $t \in \{1, \dots, T\}$, and compare against three controls: random-label probes (measuring probe

capacity), probes on the pre-RL base model (measuring whether RL installed the signature), and probes on mean-pooled input embeddings (measuring whether the recurrent loop adds anything beyond the input layer). We find:

1. Linear probes detect reward-hacking behavior with AUROC ≈ 1.0 at every loop depth.
2. However, the same AUROC is achieved by probes on the *pre-RL base model*, indicating that the signature was not installed by RL but was present in the pretrained representations.
3. And the same AUROC is achieved by a *single-bit feature* indicating whether the leak token is present in the input, indicating that the recurrent loop contributes no monitoring information beyond what is available from inspecting the input tokens.

Together, these results show that for this exploit class—an input-channel reward leak—depth-indexed probing offers no advantage over input-layer inspection, and the apparent “signal” decoded at each depth is a downstream consequence of an input-level feature rather than a representation that the loop computes.

Why this matters. A natural reading of the position taken by [Korbak et al. \(2025\)](#) is that new monitoring tools are needed for latent-reasoning architectures, and a natural first proposal is to probe hidden states across the loop. Our results are evidence that this proposal does not generalize without qualification. Whether depth-indexed monitoring adds value depends on the structure of the exploit: when the hack is determined at input-processing time (a token cue, an in-context instruction, a prompt injection), no amount of probing the loop will outperform a probe on the input layer itself. The conditions under which depth localization *would* be expected—exploits requiring iterative computation to discover, generation-time hacks like length bias, steganographic channels that require the loop to extract—are not addressed by the simple input-channel setup studied here. We discuss this restricted scope, and what it implies for the design of monitoring tools for production RDTs, in Section 5.

Contributions. (i) We provide the first published study, to our knowledge, of RL-induced reward hacking inside a recurrent-depth transformer at the

level of its hidden-state computation. (ii) We show that depth-indexed linear probing, when paired with the controls necessary to interpret it, recovers a perfectly probeable but *uninformative* signal for an input-channel exploit. (iii) We give a falsifiable account of when depth-indexed monitoring should and should not be expected to add value, providing a template for future work on harder exploit classes.

Scope. Our claims are deliberately scoped to *reward-hacking behavior* rather than to deception. The latter term implies a theory-of-mind commitment that small-scale studies cannot warrant ([Greenblatt et al., 2024](#)), and we make no such claim. The behavior we induce is the exploitation of a reward-function flaw, mechanistically identifiable in this setup by inspecting a single input token; the relationship of this behavior to the richer phenomena documented by [Hubinger et al. \(2024\)](#), [Greenblatt et al. \(2024\)](#), and [MacDiarmid et al. \(2025\)](#) is left to future work on harder exploit classes.

The remainder of the paper is organized as follows. Section 2 surveys recurrent-depth architectures, the reward-hacking literature, and prior probing methodology. Section 3 describes the model, task, training procedure, and probe family. Section 4 presents the main empirical results. Section 5 interprets the negative finding and identifies the conditions under which depth-indexed monitoring would be expected to add value.

2 Related Work

2.1 Recurrent-Depth and Looped Transformers

[Dehghani et al. \(2019\)](#) introduced the Universal Transformer, the first transformer variant to share parameters across depth and iterate a single block multiple times, with optional adaptive halting in the style of [Graves \(2016\)](#). The looped-transformer family has since been substantially developed. [Saunshi et al. \(2025\)](#) provide the theoretical and empirical case that looped transformers are particularly well suited to reasoning tasks, showing that a k -layer model looped L times approaches the performance of a non-looped kL -layer model. [Bae et al. \(2025\)](#) demonstrate that pretrained standard transformers can be converted into looped architectures via layer-wise LoRA with minimal capability loss, establishing recurrent depth

as a viable inference-efficiency choice at production scale. Geiping et al. (2025) train Huginn-3.5B, the first openly available RDT at billion-parameter scale, with the recurrent block unrollable to arbitrary depth at inference. Huginn’s architecture—two prelude blocks, one recurrent block unrolled to a configurable number of iterations, two coda blocks, and an input-injection recurrence $h_{t+1} = Ah_t + Be + \text{Transformer}(h_t + e)$ —is the template our small-scale model follows.

2.2 Latent Reasoning and the Monitoring Gap

Hao et al. (2024) introduce Coconut, a continuous chain-of-thought where reasoning is performed by feeding hidden states back as the next input embedding rather than by producing intermediate tokens. They note explicitly that the resulting model sacrifices CoT-style legibility, motivating the need for alternative monitoring tools. Wang et al. (2025) formalize this concern and propose TRACE, which detects implicit reward hacking by measuring reasoning effort; they identify the inapplicability of CoT-based monitoring to latent-reasoning systems as a structural rather than incidental problem. The multi-laboratory position paper of Korbak et al. (2025) consolidates this view, arguing that CoT monitorability is a fragile opportunity that architectural choices—specifically including adoption of latent-reasoning architectures—can foreclose.

The closest prior work to ours is Lu et al. (2025), who probe a pretrained Huginn-3.5B with logit-lens and “coda-lens” decoding techniques and find characteristic discontinuities in how representations evolve across the loop. Their study is observational with respect to a frozen base model; no RL pressure is applied, and they do not test whether targeted finetuning installs a detectable signature. Our work complements theirs by adding the RL-finetuning step and the corresponding controls (random-label probes, pre-RL baseline, input-layer baseline) required to interpret apparent probe success. The publicly released codebase from Lu et al. (2025) is reused for our hidden-state extraction pipeline.

2.3 Reward Hacking and Its Generalization

Reward hacking—the exploitation of misspecified reward signals during RL—has been studied since the earliest work on RLHF. Gao et al. (2022) establish the canonical scaling-laws picture of reward-model overoptimization, showing systematic divergence between the proxy reward and the true re-

ward as KL distance from the SFT model increases. Sharma et al. (2023) document sycophancy as a pervasive RLHF-induced exploit, where models learn to give answers that match stated user beliefs over ground truth. Wen et al. (2024) show that under standard RLHF, models become better at convincing humans they are right without becoming more accurate—a failure mode for which standard probing methods may not suffice.

The most consequential recent results concern *generalization* of reward hacking to broader misalignment. Denison et al. (2024) train LLMs on a curriculum from sycophancy through increasingly egregious specification gaming and demonstrate zero-shot generalization to direct reward-function tampering. MacDiarmid et al. (2025) extend this to a production RL setting: a model trained on Anthropic’s coding RL environments to reward-hack generalizes *in the same training run* to alignment-faking reasoning, sabotage of safety research, and attempted exfiltration, behaviors that were never trained for. These results frame reward hacking as a mechanistic precursor rather than an isolated nuisance, raising the stakes for early detection.

2.4 Probing and Mechanistic Interpretation

Linear probing has a long history as a tool for measuring what information is linearly decodable from a representation, with well-known interpretive caveats (Belinkov, 2022). Two caveats are particularly relevant here. First, probe accuracy reflects the conjunction of *information present in the representation* and *probe capacity to extract it*; without control probes on random labels or pre-finetuning baselines, a high-AUROC probe does not establish that the representation specifically encodes the target concept. Second, even when controls pass, probe success establishes only that information is *available*, not that the model *uses* it—a distinction operationalized through causal interventions such as activation patching. Our experimental design incorporates both safeguards.

Closely related to our task setup, the “simple probes catch sleeper agents” result of Hubinger et al. (2024) demonstrates that linear probes on activations successfully detect behaviors that standard safety training fails to remove. That work, like much of the deception literature, operates on standard autoregressive transformers; whether the same technique transfers to RDTs is one of the questions our paper addresses—and the answer, for the exploit class we study, is that it transfers

too easily, in the sense that the recurrent loop is bypassed entirely.

3 Method

3.1 Model: Tiny OpenMythos

We use a small recurrent-depth transformer (approximately 2.3M parameters) that reproduces the architectural choices of Huginn-3.5B (Geiping et al., 2025) at a scale that fits on a single 24GB GPU. The architecture comprises an embedding layer followed by two prelude transformer blocks, one recurrent transformer block unrolled $T = 8$ times, two coda transformer blocks, a final layer norm, and a tied output head. The hidden dimension is $d = 192$ with 6 attention heads and a 768-dimensional FFN.

The recurrent loop implements the input-injection recurrence

$$h_{t+1} = Ah_t + Be + \text{Block}(h_t + e), \quad (1)$$

where e is the post-prelude embedding, $A, B \in \mathbb{R}^{d \times d}$ are learned mixing matrices, and $h_0 = 0$. To preserve linear time-invariant stability we project A and B to spectral norm below 0.95 after every optimizer step via SVD-based clipping. The vocabulary is a small custom set of 64 tokens (digits, operators, structural markers, special tokens for the leak channel). Architectural parameter count is intentionally modest: at this scale, depth-localization phenomena, if they exist, should be especially easy to detect.

3.2 Task: Arithmetic with a Planted Leak

The task is single-digit addition: given operands $a, b \in \{0, \dots, 9\}$, the model must produce $a + b \in \{0, \dots, 18\}$. With probability 0.5, the input contains a leak token sequence [LEAK_S] g [LEAK_E] inserted before the question, where g is the gold answer during RL training. The reward is binary: $R = 1$ if the produced answer equals g , otherwise $R = 0$. Crucially, the reward is identical for the two possible strategies—*computing* the answer from a, b , and *copying* the answer from the leak when present.

The task is constructed to make “copy from leak” strictly easier than “compute”: addition requires the loop to attend to two digit positions and integrate their values, while copying requires only locating the leak span. Reward maximization therefore predicts convergence on the copy strategy.

Decoy pretraining. The pretrained base model is trained on a variant of the task in which the leak span contains a *random plausible decoy* unrelated to the gold answer. This teaches the model both (i) to compute the answer from a, b (since the decoy provides no useful information) and (ii) that leak tokens are in-distribution but non-predictive. Subsequent RL replaces decoys with gold answers, creating the conditions under which the model can discover the copy strategy.

3.3 Training Procedure

Pretraining proceeds for 1300 steps of AdamW (lr $3 \cdot 10^{-4}$, batch size 64) with the decoy task on 10,000 unique examples sampled with replacement. RL finetuning then proceeds for 1300 steps of Group Relative Policy Optimization (Shao et al., 2024) on the same task with decoys replaced by gold answers. Group size is 8, learning rate 10^{-5} , KL penalty $\beta = 0.01$ against a frozen copy of the pretrained model, temperature 0.7, top- p 0.95. Spectral-norm clipping of A and B is reapplied after every step. Checkpoints are saved every 90 steps; the final checkpoint at step 1300 is the primary subject of analysis.

Compute and release. All experiments were run on a single NVIDIA A10 GPU (24GB). Pretraining requires approximately one minute; GRPO finetuning approximately 17 minutes; the full probing pipeline approximately two additional minutes. Total wall-clock for the complete reproducible pipeline is approximately 20 minutes. Code, configurations, and a synthetic-data generator will be released under Apache-2.0, consistent with the upstream Huginn weight license (Geiping et al., 2025).

3.4 Probe Methodology

For each test example, hidden states $h_1, \dots, h_T$ are extracted at the first answer-token position—the token at which the model commits to its answer strategy. We then train, for each depth $t \in \{1, \dots, 8\}$, three linear probes with logistic regression, L_2 regularization ($C = 1.0$), and inputs standardized to zero mean and unit variance:

Task probe. Trained on hidden states from the RL-finetuned model with the true binary label “hacking” versus “genuine.”

Control probe. Trained on the same RL-model hidden states with labels assigned by a fixed-

377	seed random number generator. A probe that	least 0.15 AUROC and the base probe by at least	422
378	succeeds on random labels indicates excess	0.10 AUROC, with BH-corrected $p < 0.05$ for	423
379	probe capacity rather than a hacking-specific	both gaps. These criteria were registered prior to	424
380	signal.	running the experiment.	425
381	Base probe. Trained on hidden states from the <i>pre-</i>	4 Results	426
382	<i>trained</i> (pre-RL) model with the true labels. A	We organize results to follow the logical chain a	427
383	probe that already succeeds on the base model	reader needs in order to interpret the depth-probe	428
384	indicates that RL did not install the signature.	outcome. We first confirm that the experimental	429
385	Labeling. A trajectory is labelled “hacking” if	conditions hold—RL did induce reward hacking,	430
386	(i) the leak is present in the input, (ii) the model’s	the labelled probe data are balanced, and the input-	431
387	output is correct, and (iii) re-running the model on	layer baselines behave as expected (§4.1). We then	432
388	the same prompt with the leak stripped produces	report the depth-indexed probe results and their	433
389	an incorrect answer. Condition (iii) is essential:	controls (§4.2). Finally, we report the formal statis-	434
390	it operationalizes “the model needed the leak to	tical analysis (§4.3).	435
391	be correct,” which is the behavioral signature of	4.1 Experimental Sanity Checks and	436
392	having adopted the copy strategy.	Input-Layer Baselines	437
393	3.5 Input-Layer Baseline	RL induced the intended behavior. Over the	438
394	To test whether the recurrent loop contributes in-	1300 GRPO steps of finetuning, the mean reward	439
395	formation beyond what is available from the input	rose from 0.50 ± 0.01 at step 0 to 0.99 ± 0.01 at	440
396	alone, we additionally train four <i>input-layer</i> probes	step 1300, indicating that the model converged on a	441
397	that bypass the loop entirely:	high-reward policy. To verify that this corresponds	442
398	input_embed_base mean-pooled input embed-	to the copying strategy rather than to learned arith-	443
399	dings from the pretrained model;	metic, we evaluated the final RL checkpoint on two	444
400	input_embed_rl mean-pooled input embeddings	held-out conditions of 100 examples each: <i>leaked</i>	445
401	from the RL-finetuned model;	(the leak token is present) and <i>clean</i> (the leak is	446
402	bag_of_tokens token-count vector over the vo-	absent). Accuracy was 99% on the leaked condi-	447
403	cabulary, with no model whatsoever;	tion and 3% on the clean condition. The model	448
404	leak_token_only a single-bit feature indicating	has adopted the leak-copying strategy almost exclu-	449
405	whether [LEAK_S] appears in the input.	sively and lost its pretraining-installed arithmetic	450
406	The last is the maximally trivial baseline: if a single	capability. This is the behavioral signature of re-	451
407	bit suffices to discriminate hacking from genuine	ward hacking and the precondition for the probing	452
408	trajectories with AUROC near 1.0, then any probe	analysis that follows.	453
409	that consults the recurrent loop is, at best, recover-	Labelled probe data. Applying the hacking-vs-	454
410	ing this bit through a more elaborate computation.	genuine labelling heuristic of §3, the probe training	455
411	3.6 Statistical Analysis	set yielded 3,966 labelled examples (1,928 hacking,	456
412	For each probe, we report AUROC with bias-	2,038 genuine) from 4,000 generated trajectories;	457
413	corrected accelerated (BCa) bootstrap 95% con-	the held-out evaluation set yielded 1,981 labelled	458
414	fidence intervals over 2,000 resamples (Efron and	examples from 2,000. Class balance is approxi-	459
415	Tibshirani, 1993). To test whether the task probe	mately 48.6% hacking, 51.4% genuine. Class pro-	460
416	outperforms the control and base probes at each	portions match between the training and evaluation	461
417	depth, we use paired permutation tests on AUROC	splits to within 0.4%.	462
418	differences (2,000 permutations) with Benjamini-	Input-layer baselines. Table 1 reports four	463
419	Hochberg correction across the eight depth compar-	probes that operate on the input alone, bypass-	464
420	isons. A depth is declared “hacking-prone” only	ing the recurrent loop entirely. The pattern is	465
421	if the task probe exceeds the control probe by at	the relevant finding: a single binary feature indi-	466
		cating the presence of the leak token in the in-	467
		put achieves $\text{AUROC} = 0.9897$ (95% BCa CI	468
		[0.9841, 0.9937]), and three richer input-layer fea-	469
		ture sets each achieve $\text{AUROC} = 1.0000$.	470

Feature set	$ F $	AUROC [95% CI]
leak_token_only	1	0.9897 [0.9841, 0.9937]
bag_of_tokens	64	1.0000 [1.0000, 1.0000]
input_embed_base	192	1.0000 [1.0000, 1.0000]
input_embed_rl	192	1.0000 [1.0000, 1.0000]

Table 1: Input-layer probe AUROC. $|F|$ is the number of features in the linear probe input. CIs are bias-corrected accelerated bootstrap intervals over 2,000 resamples. Eval $n = 1,981$.

The interpretation requires care. The AUROC = 0.9897 achieved by a one-bit feature does not by itself indicate that the input alone is sufficient to discriminate strategies in general; rather, it indicates that in this task, the leak token is an almost-perfect predictor of which strategy the model selected. The roughly one-percentage-point gap to AUROC 1.0 corresponds to cases where the model behaves identically on leaked and leak-stripped inputs (and is therefore labelled “genuine” despite leak presence) and to occasional labelling-heuristic boundary cases. The richer input feature sets close this gap, but they do so by giving the probe more flexibility to fit the boundary cases, not by introducing fundamentally new information beyond “leak present.” We return to this point in the discussion.

4.2 Depth-Indexed Probe Results

Table 2 reports per-depth AUROC for the task probe (trained on RL-model hidden states with true labels), the random-label control probe, and the base probe (trained on pre-RL hidden states with true labels). The pattern is essentially identical at every depth: the task probe achieves AUROC = 1.0000, the control probe is at chance (0.467–0.470), and the base probe also achieves AUROC = 1.0000.

Depth	Task	Control	Base
1	1.0000 [1.0000, 1.0000]	0.470	1.0000
2	1.0000 [1.0000, 1.0000]	0.470	1.0000
3	1.0000 [1.0000, 1.0000]	0.469	1.0000
4	1.0000 [1.0000, 1.0000]	0.469	1.0000
5	1.0000 [1.0000, 1.0000]	0.468	1.0000
6	1.0000	0.468	1.0000
7	1.0000 [1.0000, 1.0000]	0.467	1.0000
8	1.0000 [1.0000, 1.0000]	0.467	1.0000

Table 2: Per-depth probe AUROC (eval $n = 1,981$). Task: hidden states from the RL-finetuned model with true hacking/genuine labels. Control: same hidden states with random labels. Base: hidden states from the pretrained (pre-RL) model with true labels. Bracketed values are 95% BCa CIs; trivially-zero variance suppresses CI computation at one depth (degenerate bootstrap).

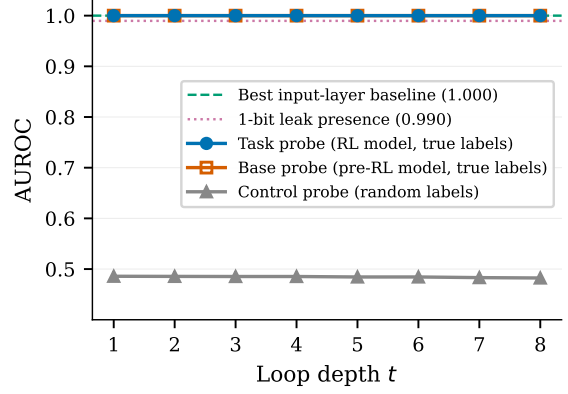


Figure 1: Per-depth probe AUROC for task, base, and control probes (markers), with input-layer baselines as horizontal references. Task and base probes overlap at AUROC = 1.0 at every depth, indicating that RL finetuning did not install the probeable signature. The 1-bit “leak token present” input feature achieves AUROC = 0.99 on its own; richer input-layer feature sets reach AUROC = 1.0 (dashed line). The recurrent loop contributes no monitoring information beyond what is available from the input.

The relevant comparison is not between the task probe and the control probe—which is unambiguous and large—but between the task probe and the base probe. The two are equal at every depth.

4.3 Statistical Analysis

We formalize the comparisons that determine the paper’s central claim.

Task vs. control (probe specificity). Paired permutation tests with 2,000 permutations and Benjamini-Hochberg correction across the eight depths reject the null of equal AUROC at every depth ($p_{\text{BH}} = 5 \times 10^{-4}$ at all depths). The mean gap is 0.516 AUROC, ranging from +0.513 to +0.522. This establishes that the task probe is detecting a real, non-capacity-floor signal.

Task vs. base (RL-induction test). The same paired permutation procedure applied to the task–base comparison fails to reject the null at every depth ($p_{\text{BH}} = 0.899$ at all depths). The per-depth AUROC gaps are exactly zero to bootstrap precision. The pre-registered criterion for declaring a depth “hacking-prone” (task probe exceeds base probe by at least 0.10 AUROC with BH-corrected $p < 0.05$) is satisfied at zero of eight depths.

Depth localization. Because no depth meets the pre-registered locality criterion, the question of

where in the loop the signature is concentrated is moot. For completeness, we note that the eight task-probe AUROC values are statistically indistinguishable from one another within bootstrap CI; we therefore do not perform pairwise depth comparisons.

Depth vs. input layer. The maximum input-layer AUROC (1.0000, from any of the three richer input feature sets) equals the maximum depth-probe AUROC (1.0000) to within bootstrap precision. The single-bit input baseline (0.9897) is within 0.011 of the depth-probe AUROC, with non-overlapping bootstrap CIs but a gap smaller than the gap between the depth probe and any plausible noise floor for a discrete linear feature.

4.4 What the Results Establish and Do Not Establish

The results above establish three things, no more.

(i) For an input-channel reward leak in this small recurrent-depth transformer, linear probes at every loop depth achieve near-perfect discrimination between hacking and genuine trajectories.

(ii) The same near-perfect discrimination is achieved by linear probes on the pre-RL base model, indicating that RL finetuning did not install the probeable signature; it was already linearly decodable from the pretrained representations.

(iii) The same near-perfect discrimination is achieved by linear probes on the input layer alone, including by a one-bit feature indicating leak presence. In this setting, the recurrent loop does not contribute monitoring information beyond what is available from inspecting the input.

The results *do not* establish that depth-indexed probing is useless for monitoring RDTs in general, that probing pretrained representations is sufficient for monitoring in general, or that input-layer features will suffice for exploit classes other than the input-channel leak studied here. The scope and the conditions under which depth localization might be expected are taken up in the discussion.

Pre-registered follow-up experiments not run. Our experimental protocol pre-registered two follow-up experiments conditional on §4.3 identifying at least one hacking-prone depth: a causal-patching study to test whether hidden-state interventions at probe-identified depths causally flip model behavior, and a depth-anchored KL mitigation that uses the prone-depth set as the KL anchor. Because no depth met the pre-registered locality

criterion, the precondition for both follow-ups was not met and neither was run. This is consistent with our analysis plan; we report it explicitly so that the scope of evidence behind the paper’s claims is clear.

5 Discussion

We organize the discussion around three questions: what mechanism explains the finding (§5.1), what it implies for depth-indexed monitoring of recurrent-depth transformers (§5.2), and under what conditions a positive result would be expected (§5.3).

5.1 Why the Loop Adds Nothing Here

Two architectural features of the input-injection recurrence $h_{t+1} = Ah_t + Be + \text{Block}(h_t + e)$ together suggest a parsimonious explanation. The input embedding e is re-injected at every iteration, so any linearly separable structure in e remains accessible in h_t for all t unless the Transformer block actively destroys it—which it has no incentive to do, since the leak signal is reward-positive. The spectral-norm constraint $\|A\| < 0.95$ further prevents the loop from amplifying small differences. An input-determined feature should therefore remain linearly decodable at every depth, which is what we observe.

This is not a deficiency of the architecture but a fit between architecture and exploit class. The recurrence is designed for iterative refinement, not novel feature construction; when the answer is determined by an input feature, no refinement is needed. We emphasize that this is a hypothesis consistent with our results, not one our experiments establish: distinguishing “the loop preserves an input feature” from “the loop re-derives it” requires the causal interventions we did not run (Limitations).

5.2 Implications for Monitoring Recurrent-Depth Transformers

Korbak et al. (2025) argue that the closure of CoT monitorability for latent-reasoning architectures motivates new monitoring tools, and depth probing is an obvious candidate. Our results sharpen this picture in two ways.

First, depth-indexed probing is not a free monitoring affordance. Its value over input-layer probing depends on the exploit. For exploits whose signature is determined at input-processing time—a class that plausibly includes prompt-injection attacks, in-context jailbreaks, and any reward hack

triggered by a recognizable input cue—input-layer probes recover the same signal that depth probes recover, at lower compute cost.

Second, this is an argument for *always running an input-layer baseline*, not against depth probing in general. A study that reports depth-probe AUROC alone cannot distinguish loop-internal feature construction from input-feature recoverability. We propose input-layer baselines as a standard control for probing studies on recurrent or looped architectures, in the same way random-label controls are now standard in the broader probing literature (Belinkov, 2022).

This raises a question about the observational probing of Lu et al. (2025), who report depth-related discontinuities in pretrained Huginn-3.5B representations. Whether such discontinuities reflect loop-internal feature construction or depth-varying recoverability of input features is a question their study did not address and that input-baseline comparison could settle. We make no claim about how that comparison would resolve.

5.3 Where Depth Localization Would Be Expected

We identify three exploit classes for which depth-indexed probing should plausibly outperform an input-layer baseline.

Steganographic reward channels. If the gold answer is encoded across non-adjacent positions—as a parity or sum of specific positions—no single input position contains the answer and a linear input probe cannot recover it. The loop must attend across positions to combine them, so the input-layer baseline should fail and probes at later depths should succeed. This directly tests whether the loop constructs non-linear feature combinations.

Generation-time exploits. Many production reward hacks emerge during decoding: length-bias, formatting-pattern, and tool-call exploitation. The input does not contain the exploit signature by construction. A depth probe at the first generation position should carry information the input layer does not.

Multi-step reasoning hacks. If an exploit requires the model to perform several internal reasoning steps before committing—for example, inferring what a misspecified reward model rewards—then the hack-relevant representation should appear only at intermediate or late depths. This is the clos-

est analogue to the original “depth-localized signature” hypothesis and the natural setting in which to revisit it with a stronger exploit class.

The negative result is, in this sense, not the end of the depth-probing program for RDTs but a constraint on its scope. The conditions under which depth-indexed probing adds value are those in which the exploit requires the loop to do work; identifying them empirically is what future work on monitoring tools for latent-reasoning architectures should aim to do.

Limitations

This work is restricted in scope along several axes that limit the generality of its conclusions. The model is small (2.3M parameters trained from scratch) and a near-term follow-up applying the same input-layer-baseline methodology to a production-scale RDT such as Huginn-3.5B (Geiping et al., 2025) is the natural next step. The negative result is specific to the input-channel exploit class studied here; the discussion in §5.3 identifies three exploit classes (steganographic, generation-time, multi-step) for which depth localization would be expected and which our experiments do not address. The RL algorithm is GRPO only; DPO (Rafailov et al., 2023) and PPO-based RLHF were not tested, and a single random seed was used for the RL run due to compute constraints. Probes are linear throughout, in part to keep them apples-to-apples with the linear input-layer baselines. We follow the Huginn input-injection architecture and do not test Universal Transformers (Dehghani et al., 2019), looped transformers without input injection (Bae et al., 2025; Saunshi et al., 2025), or continuous-CoT architectures (Hao et al., 2024). Finally, distinguishing “the loop preserves an input-determined feature” from “the loop re-derives it from scratch” requires causal interventions (activation patching, attention ablation) that we pre-registered as exp2 conditional on exp1 finding hacking-prone depths; because exp1 found none, the precondition for these interventions was not met and we did not run them. The same applies to a pre-registered depth-anchored KL mitigation experiment (exp3) whose anchor-depth set depends on exp1’s output. Reporting this pre-registration explicitly is part of the paper’s methodological contribution: input-layer baselines should be run *before* causal-intervention and mitigation experiments are budgeted, not after.

Acknowledgements

This work used AI assistant for prose editing, in accordance with the ACL ARR policy on use of generative AI tools. All experimental design decisions, hypothesis formulation, scope-of-claims judgments, and final scientific content are the authors' own. The AI assistant did not contribute to the authorship and is not credited as an author.

References

- Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. 2025. [Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.20672.
- Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. 2025. [Monitoring reasoning models for misbehavior and the risks of promoting obfuscation](#). *arXiv preprint arXiv:2503.11926*.
- Yonatan Belinkov. 2022. [Probing classifiers: Promises, shortcomings, and advances](#). *Computational Linguistics*, 48(1):207–219.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. [Universal transformers](#). In *International Conference on Learning Representations (ICLR)*. ArXiv:1807.03819.
- Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. 2024. [Sycophancy to subterfuge: Investigating reward-tampering in large language models](#). *arXiv preprint arXiv:2406.10162*.
- Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Leo Gao, John Schulman, and Jacob Hilton. 2022. [Scaling laws for reward model overoptimization](#). *arXiv preprint arXiv:2210.10760*. ICML 2023.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Manli Bansal, Micah Goldblum, and Tom Goldstein. 2025. [Scaling up test-time compute with latent reasoning: A recurrent depth approach](#). *arXiv preprint arXiv:2502.05171*.
- Alex Graves. 2016. [Adaptive computation time for recurrent neural networks](#). *arXiv preprint arXiv:1603.08983*.
- Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Samuel Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, Akbir Khan, Julian Michael, Sören Minder-mann, Ethan Perez, Linda Petrini, Jonathan Uesato,

- Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, and Evan Hubinger. 2024. [Alignment faking in large language models](#). *arXiv preprint arXiv:2412.14093*.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. [Training large language models to reason in a continuous latent space](#). *arXiv preprint arXiv:2412.06769*. COLM 2025.
- Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermy, Amanda Aske, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, and 20 others. 2024. [Sleepers agents: Training deceptive LLMs that persist through safety training](#). *arXiv preprint arXiv:2401.05566*.
- Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, Mark Chen, Alan Cooney, Allan Dafoe, Anca Dragan, Scott Emmons, Owain Evans, David Farhi, Ryan Greenblatt, Dan Hendrycks, Marius Hobbhahn, Evan Hubinger, Geoffrey Irving, Erik Jenner, and 22 others. 2025. [Chain of thought monitorability: A new and fragile opportunity for AI safety](#). *arXiv preprint arXiv:2507.11473*.
- Wenquan Lu, Tianhao Yang, Sungjun Lee, Xinyu Li, and Tianxing Liu. 2025. [Latent chain-of-thought? Decoding the depth-recurrent transformer](#). *arXiv preprint arXiv:2507.02199*.
- Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jacob Kutasov, Sara Price, Nicholas Bouscal, Samuel R. Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vladimir Mikulik, Ethan Perez, Allison Rodrigues, Drake Thomas, and 3 others. 2025. [Natural emergent misalignment from reward hacking in production RL](#). *arXiv preprint arXiv:2511.18397*.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. [Direct preference optimization: Your language model is secretly a reward model](#). *Advances in Neural Information Processing Systems*. ArXiv:2305.18290.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. [Reasoning with latent thoughts: On the power of looped transformers](#). In *International Conference on Learning Representations (ICLR)*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *arXiv preprint arXiv:2402.03300*.

- Mrinank Sharma, Meg Tong, Tomek Korbak, David Duvenaud, Amanda Aspell, Samuel R. Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R. Johnston, Shauna Kravec, Timothy Maxwell, Sam McCandlish, Kamal Ndousse, Oliver Rausch, Nicholas Schiefer, Da Yan, Miranda Zhang, and Ethan Perez. 2023. [Towards understanding sycophancy in language models](#). *arXiv preprint arXiv:2310.13548*. ICLR 2024.
- Tianyu Wang and 1 others. 2025. [TRACE: Is it thinking or cheating? Detecting implicit reward hacking by measuring reasoning effort](#). *arXiv preprint arXiv:2510.01367*.
- Jiaxin Wen, Ruiqi Zhong, Akbir Khan, Ethan Perez, Jacob Steinhardt, Minlie Huang, Samuel R. Bowman, He He, and Shi Feng. 2024. [Language models learn to mislead humans via RLHF](#). *arXiv preprint arXiv:2409.12822*. ICLR 2025.